

1. **Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.**

AIM: Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

ALGORITHM:

1. Include necessary headers: Include the necessary header files like `<stdio.h>` and `<unistd.h>` for using system calls.
2. Declare variables: Declare variables to hold the process ID (`pid_t pid`) and the parent process ID (`pid_t ppid`).
3. Get the current process ID: Use the `getpid()` system call to retrieve the process ID of the current process.
4. Get the parent process ID: Use the `getppid()` system call to retrieve the parent process ID.
5. Display the process IDs: Print the process IDs to the console.
6. Create a new process: Use the `fork()` system call to create a new process. Check the return value of `fork()` to determine whether the code is running in the parent or child process.
7. Display process IDs in child and parent processes: Depending on whether the process is the parent or the child, display the respective process IDs.

PROGRAM:

```
#include<stdio.h>
#include<unistd.h>
int
main()
{
    printf("Process ID: %d\n", getpid() ); printf("Parent
    Process ID: %d\n", getpid() );return 0;
}
```

```
C:\Users\jaswa\OneDrive\Doc x + v
Process ID: 31556
Parent Process ID: 31556

-----
Process exited after 0.05791 seconds with return value 0
Press any key to continue . . . |
```

2. Identify the system calls to copy the content of one file to another and illustrate the same using a C program

AIM: Identify the system calls to copy the content of one file to another and illustrate the same using a C program

ALGORITHM:

- Include necessary headers:
- Include the necessary header files like <stdio.h>, <fcntl.h>, and <unistd.h> for working with file-related system calls.
- Declare variables:
- Declare variables to hold file descriptors, buffer, and other necessary information.
- Open the source and destination files:
- Use the open system call to open the source and destination files, obtaining file descriptors for each.
- Read from the source file:
- Use the read system call to read data from the source file into a buffer.
- Write to the destination file:
- Use the write system call to write the data read from the source file into the destination file.
- Close the files:

Use the close system call to close the source and destination files.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    FILE *fptr1, *fptr2; char
    filename[100], c;

    printf("Enter the filename to open for reading \n");
    scanf("%s", filename);
    fptr1 = fopen(filename, "r"); if
    (fptr1 == NULL)
    {
        printf("Cannot open file %s \n", filename);
        exit(0);
    }
}
```

```

printf("Enter the filename to open for writing \n");
scanf("%s", filename);
fptr2 = fopen(filename, "w");if
(fptr2 == NULL)
{
    printf("Cannot open file %s \n", filename);
    exit(0);
}
c = fgetc(fptr1);
while (c != EOF)
{
    fputc(c, fptr2); c
    = fgetc(fptr1);
}

printf("\nContents copied to %s", filename);

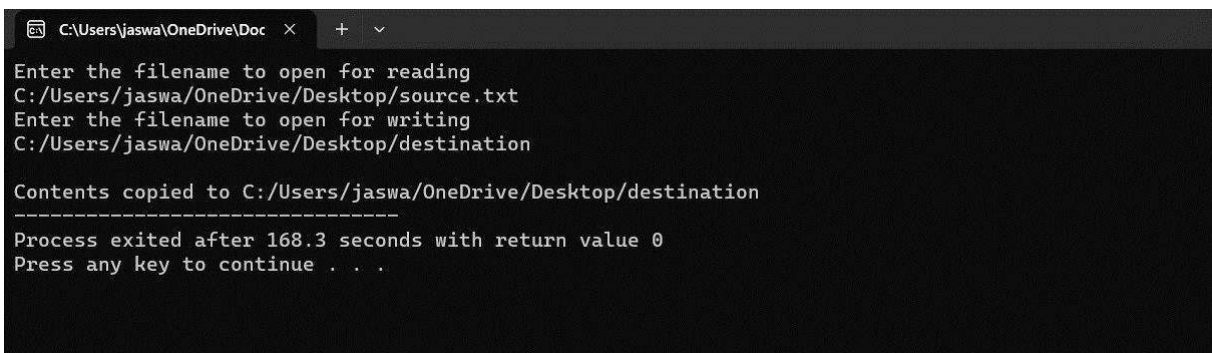
fclose(fptr1);
fclose(fptr2);
return 0;

}

```

OUTPUT

3. Design a CPU scheduling program with C using First ComeFirst Served



```

C:\Users\jaswa\OneDrive\Doc >
Enter the filename to open for reading
C:/Users/jaswa/OneDrive/Desktop/source.txt
Enter the filename to open for writing
C:/Users/jaswa/OneDrive/Desktop/destination

Contents copied to C:/Users/jaswa/OneDrive/Desktop/destination
-----
Process exited after 168.3 seconds with return value 0
Press any key to continue . . .

```

technique with the following considerations.

- a.** All processes are activated at time 0.
- b.** Assume that no process waits on I/O devices.

AIM: Design a CPU scheduling program with C using First ComeFirst